LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN
INSTITUT FÜR INFORMATIK
LEHRSTUHL FÜR DATENBANKSYSTEME
UND DATA MINING



Practical Report: Stream Clustering
in Computer Science (M.Sc.)

Experiencing Stream Clustering – Endless Waves of Data

Felix Hajuj

Mentors: Philipp Jahn, Dr. Gabriel Marques Tavares
Submission Date: 01.08.2026

Abstract

This report documents my personal journey through the practical course *Practical Stream Clustering* at LMU Munich (Summer Semester 2026). Over the course of four algorithmic blocks, working with different teams, we implemented and evaluated a progression of stream clustering algorithms: from the grid-based **DStream** (Block 0), through the micro-cluster approach of **DenStream** (Block 1), the pure-online **CEDGM** (Block 2), to the neural-network-inspired **DenSOINN** (Block 3) and finally the anytime concept with **LiarTree** (Block 4). Each block involved reading the original research paper, implementing the algorithm within a shared Python framework, evaluating it on real datasets, and presenting the results. This report reflects my individual contributions to the implementation, the challenges I encountered, the design decisions I made, and the key insights I took away from each algorithm.

Contents

1 Introduction **2**

2 Block 0: DStream — My First Stream Clusterer **4**
2.1 Algorithm Overview 4
2.2 Implementation 4
2.3 Challenges and Design Decisions 5
2.4 Evaluation and Observations 6

3 Block 1: DenStream — Micro-Clusters over Evolving Streams **7**
3.1 Algorithm Overview 7
3.2 My Contribution: The MicroCluster Class 8
3.3 Challenges and Design Decisions 9

4 Block 2: CEDGM — Pure Online Clustering with a Grid **10**
4.1 Algorithm Overview 10
4.2 My Contribution: Core Architecture 11
4.3 Challenges, Design Decisions & Observations 12

5 Block 3: DenSOINN — A Neural Approach to Stream Clustering **13**
5.1 Algorithm Overview 13
5.2 My Contribution: Offline Phase and Evaluation 14
5.3 Challenges, Design Decisions & Observations 16

6 Block 4: LiarTree — Anytime Clustering with Noise **19**
6.1 Algorithm Overview 19
6.2 My Contribution: Noise Handling 20
6.3 Challenges, Design Decisions & Observations 21

Bibliography **25**

Chapter 1

Introduction

Modern data rarely sits still. Sensor networks, financial tickers, social media feeds, and industrial telemetry all produce data points in a continuous, potentially infinite flow. Traditional clustering algorithms such as K-Means [7] or DBSCAN [4] assume the entire dataset is available in memory upfront — an assumption that immediately breaks down in this setting. Stream clustering algorithms address this by processing each data point once, on arrival, while maintaining a compact in-memory summary of what has been seen so far.

What makes stream clustering fundamentally harder than batch clustering is a set of properties that any viable algorithm must respect:

- **Infinite stream** — data arrives continuously with no defined end, so algorithms must process each point on arrival without waiting for the full dataset.
- **Limited processing time** — each incoming point must be handled quickly; expensive operations that scale with dataset size are not viable.
- **Bounded memory** — raw data points must be discarded after processing and replaced by compact summaries (*compression*); random access to past points is impossible (*single scan*).
- **Evolving distribution (concept drift)** — the underlying data distribution can shift over time, requiring models to age and forget stale information continuously.
- **Noisy data** — real streams contain spurious observations that must be identified and excluded from the cluster structure.
- **Varying data rates** — algorithms must remain correct under peak load, adapting to the fastest possible arrival rate.

This report is structured into four blocks preceded by a solo pre-task:

- **Block 0 (Pre-task):** DStream [3] — a grid-based Online-Offline algorithm
- **Block 1:** DenStream [2] — micro-cluster-based Online-Offline clustering
- **Block 2:** CEDGM [8] — a pure-online, grid-indexed hybrid approach
- **Block 3:** DenSOINN [1] — a neural-network-based stream clusterer
- **Block 4:** LiarTree [6] — anytime clustering

For each block, we read the original research paper, implemented the algorithm within a shared Python framework (the *baseframework*), and evaluated it on different datasets: `complex9` (nine non-convex shapes), `rbf3` (three Gaussian blobs), `HAR` (human activity recognition, high-dimensional), and `stream_densired` (a synthetic stream with concept drift) as well as other datasets found online which some papers have used as well. Clustering quality was measured mainly with two metrics: the Cluster Mapping Measure (CMM) and the Temporal Silhouette Index (TS-Index). But sometimes we used additional metrics in order to be able to compare to the results acheived in the original papers.

Chapter 2

Block 0: DStream — My First Stream Clusterer

2.1 Algorithm Overview

DStream [3] is a grid-based Online-Offline stream clustering algorithm. The core idea is simple: instead of tracking individual data points, the feature space is divided into uniform grid cells, and each cell maintains a single scalar *weight* that counts how much data has landed in it, this is discounted over time using exponential decay:

$$w_{\text{new}} = w_{\text{old}} \cdot \lambda^{\Delta t} + 1 \tag{2.1}$$

where $\lambda \in (0, 1)$ is a decay factor and Δt is the number of time steps since the cell was last updated. This aging mechanism is what handles concept drift: old data naturally fades and stops influencing the current cluster structure.

Each cell is assigned one of three statuses based on its weight relative to two fixed thresholds: **DENSE**, **TRANSITIONAL**, or **SPARSE** (noise). DStream also has a cleanup routine that scans all cells and removes ones that have decayed below a minimum density, using a two-step grace period to avoid early deletion. The offline phase then groups adjacent dense and transitional cells into macro-clusters using a simple adjacency clustering step. An example of DStream and the macro clusters it found can be seen in Figure 2.1.

2.2 Implementation

This was a solo pre-task, so I implemented the full algorithm from scratch. The main building blocks are a `GridCell` class that holds the cell weight, last-update timestamp (`t_g`), last-deletion timestamp (`t_m` for the grace period),

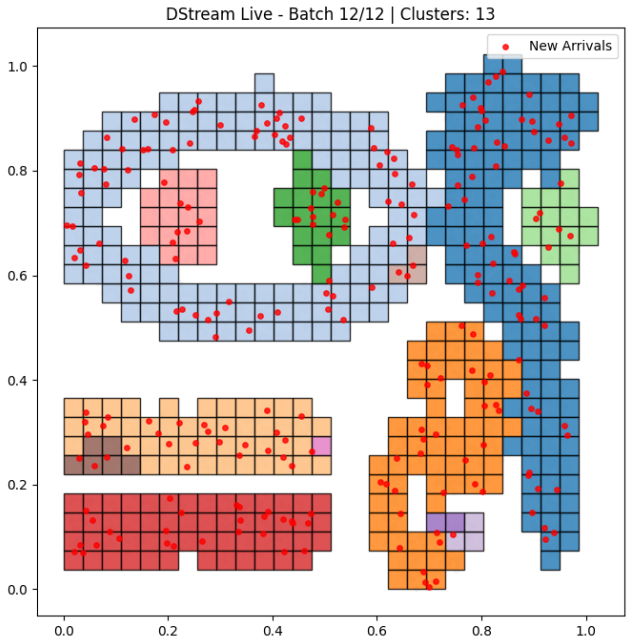


Figure 2.1: DStream Example on Complex9.

current `Status`, and a `marked_sporadic` flag for the two-step deletion process. `Status` is a small Python `Enum` with three values: `DENSE`, `TRANSITIONAL`, and `SPARSE`.

The online phase (`DStreamOnlinePhase`) maps each incoming point to its cell index via `np.floor(x / grid_size)` and updates the cell weight in-place. Cleanup fires every `cleanup_interval` steps, scanning all cells and removing those that are both below the density threshold and past their grace period. The offline phase (`DStreamOfflinePhase`) converts the surviving cell indices to actual coordinates, then delegates to the baseframework's `adjacency_clustering` function to form macro-clusters from spatially adjacent cells. A thin `DStream` wrapper ties both phases together and implements the baseframework's `learn` / `predict` interface. The entry point `run.py` adds an `argparse` CLI so all parameters (grid size, decay factor, thresholds, batch size) can be tuned from the command line without touching source code.

2.3 Challenges and Design Decisions

The DStream algorithm is quite straightforward and acts as a good entry-level to understand the main concepts of stream clustering, since I could visualize the gridcells and the connections to create the final macro-clsuters. One in-

teresting remark here is how we handle *cleanup logic*. The paper describes a two-step deletion where a cell is first flagged as sporadic and only removed if it is still below threshold at the next inspection, unless it received a new data point in between. Translating that into a clean stateful loop required tracking `marked_sporadic` and `t_m` per cell and reasoning carefully about which checks happen in which order.

I used the following **adjacency threshold** in the offline phase: `(grid_size / 2) * 1.1`, it acts as the distance cutoff for two cells to be considered adjacent (a slight margin above the exact cell-center distance between neighbors). Getting this wrong causes fragmentation: clusters that should merge stay split into many small components, which reduces the CMM score immediately. This made me realize how sensitive grid-based methods, and stream-clustering in general, are to their hyperparameters: the same algorithm can look completely broken or quite clean depending on a single floating-point number.

One subtler issue was the **boundary at weight = 1.0**. A cell that receives exactly one data point gets weight exactly 1.0 (since $0 \cdot \lambda^{\Delta t} + 1 = 1$). If the sparse threshold is set at or above 1.0, freshly populated cells are immediately marked as noise and cleaned up before they can accumulate more data, this does not crash anything but quietly stops clustering from working.

2.4 Evaluation and Observations

On `complex9` and `rbf3`, DStream produced sensible clusters once grid size was tuned to the data's natural scale. Grid cells are an incredibly cheap summary — just a weight and a timestamp — keeping the online phase extremely fast. The downside is that cluster shapes are constrained by cell granularity: non-convex clusters require a fine grid, but finer grids explode cell count and memory. Fragmentation was the recurring problem in early experiments, where many small fragments matched the same ground-truth class, inflating the predicted k while CMM scores looked deceptively high. As the opening algorithm of the course, DStream was a good warm-up: every design decision has an immediate and visible effect on the output.

Chapter 3

Block 1: DenStream — Micro-Clusters over Evolving Streams

3.1 Algorithm Overview

DenStream [2] is widely considered one of the landmark papers in stream clustering. Before it, most stream clustering algorithms either required knowing the number of clusters in advance (like k-means variants) or were restricted to convex cluster shapes. DenStream broke both constraints by combining a density-reachability model (borrowed from DBSCAN [4]) with a streaming-compatible summary structure, enabling arbitrary-shaped, noise-aware clustering without ever seeing the full dataset.

The algorithm moves away from the rigid grid of DStream and instead tracks the data distribution through floating *micro-clusters*. Small spherical summaries that can sit anywhere in the feature space and shift as new data arrives.

Each micro-cluster maintains a CF-tuple $(w, \mathbf{CF1}, \mathbf{CF2})$: scalar weight, per-dimension linear sum, and per-dimension squared sum. Center $(\mathbf{CF1}/w)$ and radius $(\sqrt{\sum(\mathbf{CF2}/w - (\mathbf{CF1}/w)^2)})$ are recoverable in $O(1)$. Because the fading factor $f(\Delta t) = 2^{-\lambda\Delta t}$ scales all three fields proportionally, it cancels in the division — decay reduces a cluster's weight for pruning without shifting its center or radius.

DenStream distinguishes between **Potential Micro-Clusters** (PMCs), which have accumulated enough weight to be credible cluster summaries, and **Outlier Micro-Clusters** (OMCs), which are new or low-weight and treated as tentative. When a data point arrives, the online phase tries to merge it into the nearest PMC that keeps the radius below ε; if none fits, it falls back

to OMCs; if that also fails, a new OMC is created. A periodic cleanup step prunes OMCs that can provably never accumulate enough weight to become PMCs. The offline phase then runs DBSCAN on the PMC centers to form macro-clusters, this workflow is demonstrated in Figure 3.1.

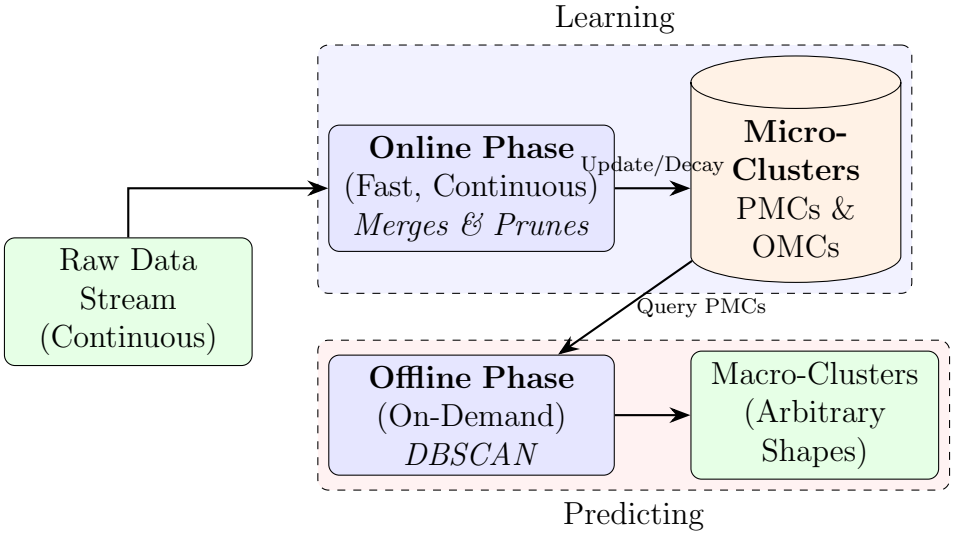


Figure 3.1: The two-phase workflow of the DenStream algorithm.

3.2 My Contribution: The MicroCluster Class

This block was a group of four: **Anton** handled the routing engine (`learn_one`), **Nils** built the offline phase and CLI, and **Jack** implemented the periodic cleanup and worked on evaluation. **I** took more of an architectural role here, my contribution was the initial skeleton of the whole algorithm that set up the file structure following the base-framework and guided my team, plus the `MicroCluster` class itself, and the visualization.

The `MicroCluster` class stores `weight`, `CF1`, `CF2` plus a creation timestamp `t_0` and `last_update` for lazy decay tracking. `insert_sample` updates all three fields in one vectorized operation; `apply_decay` scales them by $2^{-\lambda\Delta t}$; `get_center` and `get_radius` derive the geometry in $O(1)$; and `copy` produces a deep copy so the routing engine can simulate a merge before committing it.

One implementation decision worth noting is that `CF1` and `CF2` are stored as NumPy arrays (`np.zeros_like(point)`), not as Python lists or scalars. This means `insert_sample` and `apply_decay` are fully vectorized: `CF1 += point` and `CF2 *= fading_fraction` operate across all dimensions in a single C-level loop. For a 2D dataset this barely matters, but for high-dimensional data like

`HAR` (561 features), this vectorization avoids a Python-level per-dimension loop and keeps the online phase within acceptable runtime.

Alongside the MicroCluster class, I extended the Jupyter playground notebook that I had originally built for DStream. For DenStream I added a step-by-step visualizer that renders the live micro-cluster state after each batch: each PMC is drawn as a circle with its actual radius, OMCs shown in grey, and the macro-cluster coloring updated after each offline query. This was genuinely useful for the whole team during development, since it made it immediately obvious when a parameter was wrong (e.g., ε too large merges everything into one bubble, visible at a glance) rather than having to inspect raw metric values.

3.3 Challenges and Design Decisions

The biggest conceptual shift from DStream is that micro-clusters have no fixed grid key, requiring integer IDs instead of cell-index tuples — a small change that ripples through how the rest of the system references and updates clusters.

The most important design decision was `copy()`. The routing engine simulates a merge before committing it: it copies the micro-cluster, inserts the point, and checks whether the resulting radius stays below ε. Without a true deep copy of the NumPy arrays, this silently mutates the original — a bug that only shows up as gradually bloating clusters. Two easy-to-miss correctness details: decay must apply to all three CF fields (not just weight), otherwise the center drifts while the weight fades; and the radius formula needs a `max(0, ...)` guard against floating-point rounding below zero.

A practical design choice our team made was **lazy decay**: instead of reapplying the fading function to every micro-cluster after every incoming point (which would be $O(N \cdot |\text{MCs}|)$), each MC only decays when it is actually accessed. The `last_update` field tracks when the last decay was applied, so the correct cumulative factor can be computed on demand. Same mathematical result, drastically lower computation cost per point.

The **main takeaway** for me was that the CF-tuple is a surprisingly elegant data structure: three arrays give you the center, radius, and total mass of an arbitrary-shaped point cloud, all updateable in $O(d)$ per point, and all queryable in $O(1)$. The online phase is fast precisely because it never touches raw data again after absorption. The difficulty is that the algorithm's quality depends heavily on the its hyperparameters, meaning domain knowledge is very valuable. Early stream behaviour and the order of the data also plays a big role, because the first clusters to form shape everything that follows.

Chapter 4

Block 2: CEDGM — Pure Online Clustering with a Grid

4.1 Algorithm Overview

CEDGM [8] introduces the next paradigm shift in this course: a **pure online** algorithm that maintains a live, always-queryable clustering without any separate offline phase. This is a big step up from DStream and DenStream, where macro-clusters only exist after an explicit offline trigger. In CEDGM every incoming point immediately updates the cluster state, and the current macro-cluster assignment is always available, as described in Figure 4.1

Despite the name "density Grid-based Method," CEDGM is actually a *hybrid* algorithm, and this naming was one of the first things that confused me when reading the paper. The grid is *not* the cluster representation. It is a pure spatial index: an optimization step that answers the question "which micro-clusters are nearby?" in $O(1)$ instead of scanning all of them. The actual summaries are still floating micro-clusters (CMCs and OMCs) with a center and a radius, just like in DenStream.

The core structure is an **AdjacencyGraph** that runs alongside the grid. Nodes are CMCs; two CMCs are connected if their regions overlap. The connected components of this graph are the macro-clusters. This graph is maintained *incrementally*: every time a CMC is created, moved, or deleted, the graph updates locally; no full recompute. This is what makes real-time clustering possible without a periodic offline step.

Each micro-cluster in CEDGM additionally has a `life` field, a survival timer that is boosted whenever a point is absorbed and drained linearly over time by a decay factor. A CMC whose life reaches zero is pruned. This is different from the proportional exponential fading in DenStream, becuase here we separate the weight and the life of a micro cluster.

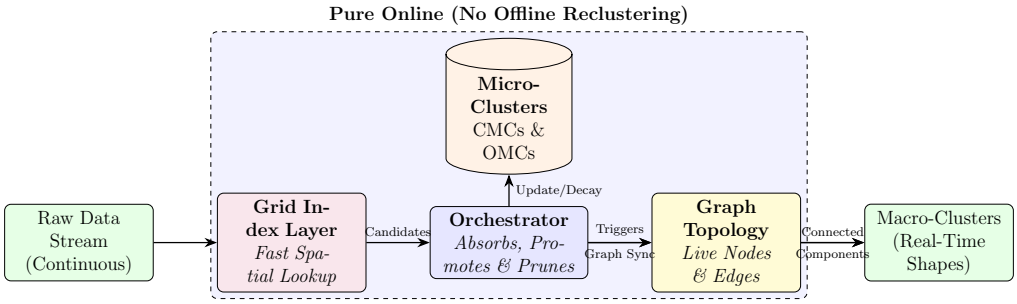


Figure 4.1: The workflow of the CEDGM algorithm.

4.2 My Contribution: Core Architecture

This block was a group of three: **Paul** implemented the full graph logic; **Anton** handled merging and pruning details + extending the visualization notebook into its own tool. **I** built most of the structural scaffolding that the rest of the system was built on.

GridSpace (`grid.py`) is the spatial index layer. The key method is `get_neighboring_cells`, which returns all 3^d cells in the neighborhood without hard-coding 2D offsets — it uses `itertools.product` over `(idx-1, idx, idx+1)` per dimension, generalizing automatically. Cell membership is tracked in $O(1)$ via Python sets through `add/remove/move_mc_between_cells`.

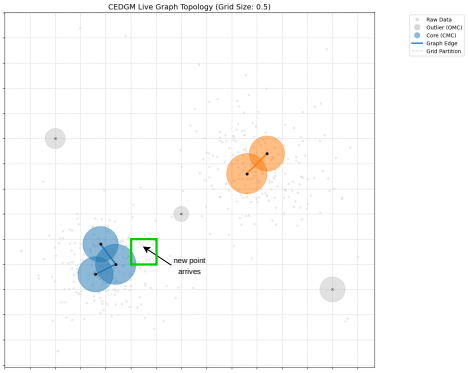


Figure 4.2: Assigning new point to a grid-cell.

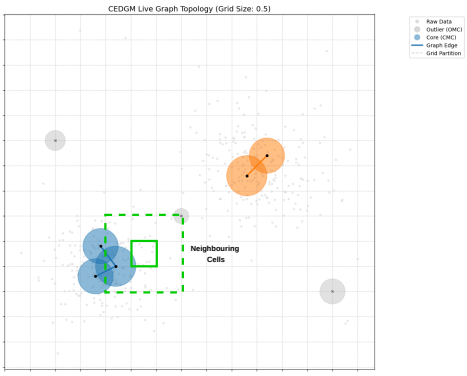


Figure 4.3: Select neighbour and fetch CMCs and OMCs → reduce dist. calculations.

MicroCluster (`micro_clusters.py`, CEDGM version) shares the CF-tuple core from DenStream but adds two CEDGM-specific attributes: `density` (running count of absorbed points) and `life` (survival timer). There is also a kernel/shell radius distinction: `get_kernel_radius` returns the exact variance-

based radius, while `get_shell_radius` multiplies it by a factor of 1.5 to define the broader overlap zone used for adjacency checks. A point landing in the kernel does not shift the center (the center is considered stable), while a point landing in the shell does. This is controlled by the `move_center` flag in `insert_sample`.

I also designed the initial **CEDGM orchestrator** (`cedgm.py`), laying out the per-point decision tree: look up candidates in neighboring cells → find nearest CMC and OMC → merge, promote, or create → update grid cell membership if center moved → notify graph.

4.3 Challenges, Design Decisions & Observations

The biggest initial confusion was the naming: I spent time trying to understand how a "grid-based" method could produce arbitrary cluster shapes, before realizing the grid is only a lookup index. Once I understood that, the architecture became clear.

The trickiest detail was **tracking center drift**. When a CMC absorbs a shell-region point its center shifts, and if the shift crosses a cell boundary the GridSpace membership must be updated immediately — otherwise the candidate lookup silently misses the CMC for future points. This coupling between floating centers and a discrete index is something DStream never had to deal with.

The **real-time graph consistency** requirement compounded this: in DStream and DenStream the macro-cluster structure is only needed at query time, so errors surface clearly. In CEDGM the graph must be correct after every single `learn_one` call, because every call returns a live cluster ID. Any missed edge update after a center shift produces silently wrong labels — far harder to debug than an incorrect offline result.

The grid-based candidate lookup also turned out to be one of the clearest wins compared to DenStream: instead of scanning all micro-clusters, only the (3^d) neighboring cells are checked.
Althoug the paper was missing some details regarding their approach, I still found this algoirthm very interesting and my main personal takeaway from this block is that the real cost of pure online clustering is not the algorithm itself but the infrastructure around it; the spatial index, the real-time graph, the drift detection. These are what keep the system correct at every time step, and they are what made this block the most architecturally complex one so far.

Chapter 5

Block 3: DenSOINN — A Neural Approach to Stream Clustering

5.1 Algorithm Overview

DenSOINN [1] (**Den**sity-based **S**elf-**O**rganizing **I**ncremental **N**eural **N**etwork) moves away from the grid-cell and micro-cluster summaries of the previous blocks and replaces them with a graph $G = \langle V, E \rangle$ whose topology is learned directly from the stream. Each node v_i is a micro-cluster prototype: a centroid w_i and an accumulated count m_i. The graph is all that persists in memory.

The **online phase** uses Competitive Hebbian Learning. For each point the two nearest nodes (s_1, s_2) compete using a *self-adaptive metric*: a diagonal matrix $A_{ii} = 1/(\max_i - \min_i)^2$ updated each round from current node positions — equivalent to Euclidean on min-max normalized data without ever normalizing, making DenSOINN the first algorithm in this course that handles raw, unscaled data natively. Absorbed points reinforce the s_1–s_2 edge; unabsorbed points spawn new nodes. Periodic denoising removes low-weight isolated nodes, leaving a topological skeleton of the data.

The **offline phase** clusters this skeleton in three steps: BFS to find connected components ("islands"), density peak scoring per component to locate cluster centers ("mountaintops"), and a downhill cascade to label the remaining nodes. The workflow is shown in Figure 5.1.

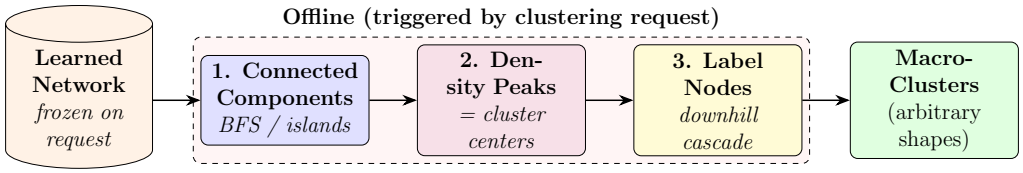


Figure 5.1: The three-stage offline phase of DenSOINN.

5.2 My Contribution: Offline Phase and Evaluation

This block was a group of three: **Paul** built the full online phase (Competitive Hebbian Learning loop, network data structure, metric update); **Anton** provided the initial skeleton and built the interactive visualization tool, such that it's running on its own domain now under: `https://stream-viz.birdiegang.uk/`. **I** took on the offline clustering phase and the evaluation pipeline.

Offline Phase: The Islands–Mountains View

The offline phase receives a `summaries` snapshot mapping each node to its centroid, weight, and neighbor set. My first step is always to **rebuild the self-adaptive metric from the frozen centroids**: without this, downstream distances fall back to plain Euclidean, breaking consistency with how the online phase built the graph.

1. Connected components (`_get_components_from_summaries`) are found via BFS over the neighbor lists in the summary dict. The offline phase does zero active cutting here, it trusts the online phase, which already severed weak edges and removed noise nodes during denoising:

```
queue = [node]; visited.add(node)
while queue:
    curr = queue.pop(0)
    for nbr in summaries[curr]["neighbors"]:
        if nbr not in visited and nbr in summaries:
            visited.add(nbr); queue.append(nbr)
```

Each component is then treated independently, guaranteeing that clusters physically separated in feature space are never accidentally merged.

2. Density peak scoring (`_find_cluster_centers`) is where the "mountaintops" are located. For each node i I compute a local density:

$$p_i = \frac{m_i}{1 + d_i^{n'}} \tag{5.1}$$

where m_i is the node weight, d_i is the mean Mahalanobis distance to its topological neighbors, and n' is the intrinsic dimensionality parameter ($n' = 2$ by default). Nodes are sorted by density descending. For each non-peak node i, I find its nearest denser neighbor q_i and compute a peak score:

$$h_i = p_i \cdot d_A(w_i,\, w_{q_i}) \tag{5.2}$$

A node becomes a cluster center if $h_i \geq \alpha \cdot \bar{h}$, where $\bar{h}$ is the mean peak score over non-peak nodes. Figure 5.2 shows this visually: peaks are nodes that are simultaneously dense (high p_i, tall mountain) and isolated from any denser region (large $d(w_i, w_{q_i})$, far from higher ground).

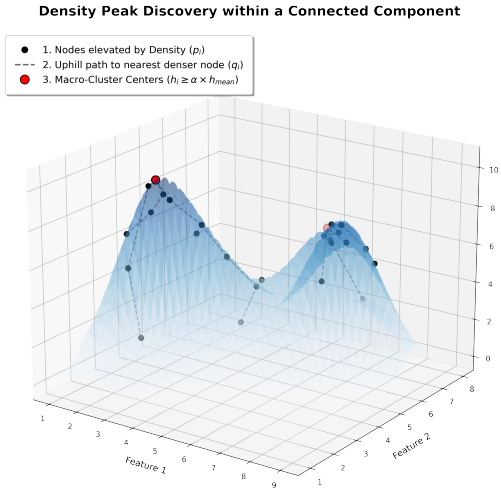


Figure 5.2: Density peak discovery within a connected component.

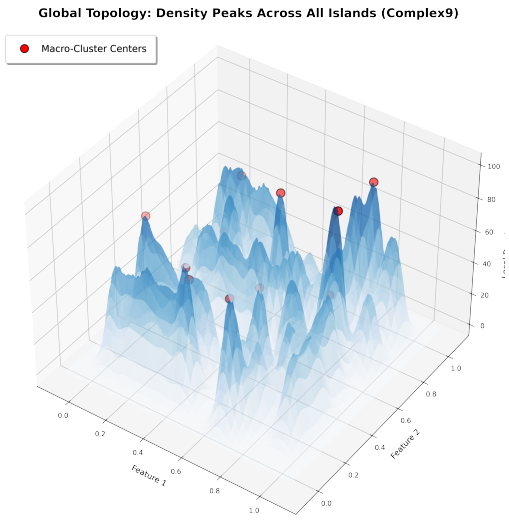


Figure 5.3: Islands and Mountains Global View of Complex9

3. The downhill cascade propagates labels from peaks outward: each unlabeled node inherits the label of its nearest denser neighbor q_i, following the topology downhill until reaching a labeled center. I also implemented a `predict_summaries_with_stages` variant exposing intermediate state at each stage, which was essential for Anton's visualization tool.

Evaluation

Feature scale disparity analysis: for the evaluation I first computed two scale-disparity metrics across our datasets: the max range ratio and the max variance ratio, which measure how much the widest and highest-variance feature dominate over the narrowest and lowest-variance one. The results, shown

in Table 5.1, told a clear story about where DenSOINN's adaptive metric would matter:

Table 5.1: Feature scale disparity across datasets. Covertype is the stress test.

Dataset	**Max Range Ratio**	**Max Variance Ratio**
RBF3	1.00	1.31
Spiral	1.11	1.02
Stream Densired	1.12	1.02
Complex9	1.59	2.83
HAR	1.00	407.49
Covertype	7.17×10^3	4.70×10^{11}

Covertype benchmark: to evaluate DenSOINN's core claim, I ran three configurations on a 50k sample of Covertype (581k instances, 54 attributes, 7 classes, variance ratio 4.70×10^{11}) and compared against sklearn's K-Means. Results across 10 randomized runs are shown in Table 5.2:

Table 5.2: Covertype benchmark: DenSOINN vs. K-Means (50k sample). K-Means raw collapses; DenSOINN raw matches the paper.

Algorithm	**Input**	**NMI**	**ARI**	**CMM**	**TS**
K-Means (sklearn)	Scaled (50k)	0.162	0.085	0.879	0.388
K-Means (sklearn)	Raw (50k)	0.078	0.012	0.869	0.379
DenSOINN	Scaled (50k)	0.151 ± 0.028	0.033 ± 0.030	$\mathbf{0.847 \pm 0.021}$	$\mathbf{0.440 \pm 0.031}$
DenSOINN	Raw (50k)	$\mathbf{0.161 \pm 0.012}$	$\mathbf{0.053 \pm 0.020}$	0.839 ± 0.016	-0.117 ± 0.050
DenSOINN (paper)*	Raw	0.146 ± 0.029	0.028 ± 0.031	—	—

* Paper ran 10 randomized runs over the full 581k dataset.

5.3 Challenges, Design Decisions & Observations

Intrinsic dimensionality n' was the first issue I had to reason through carefully. The density formula uses $d_i^{n'}$ as a proxy for the volume of the neighborhood around a node. In the ambient space, volume scales as d^D where D is the number of features — for HAR's 561 dimensions, any reasonable distance value raised to the 561st power either collapses to zero or explodes. The fix is the manifold hypothesis: real data rarely spans all dimensions freely. A dataset of human motion recordings may have 561 sensor channels but only a handful of meaningful degrees of freedom (speed, direction, activity type).

Setting $n' = 2$ keeps the density math stable regardless of ambient dimension, and empirically it worked well across all our datasets including HAR (variance ratio 407).

The global peak edge case required careful reading of the paper. The density peak algorithm says: compute h_i for all nodes, threshold by $\alpha \cdot \bar{h}$. But the single densest node in each component has no denser neighbor — its q_i is undefined. The paper implies $h = \infty$ for this node, meaning it always seeds a cluster. In the code I handle this by extracting `sorted_nodes[0]` before computing $\bar{h}$ and assigning it label 0 unconditionally, then computing $\bar{h}$ only over the remaining nodes. Getting this wrong produces silent label-propagation bugs where an entire component inherits a stale label.

Metric rebuild in offline. The online phase updates the metric from live `Node` objects; the offline phase only has the summary dict. I added `update_from_centroids` to reconstruct A from the frozen centroid matrix. Without it, peak scoring uses plain Euclidean distances, which can rank peaks very differently on high-variance data.

The TS/CMM split on Covertype raw was the result I found most instructive. CMM stays strong at 0.839 on raw data, but Temporal Silhouette drops to -0.117. This is not a bug: TS measures separation in raw Euclidean space, which the self-adaptive metric has already warped. Clusters well-separated in Mahalanobis space can look interleaved in the original space, so TS reports negative separation while CMM (label-based) stays high. The lesson: choosing a distance metric also implicitly chooses which evaluation metrics are meaningful. K-Means on the same raw data collapses from NMI 0.162 (scaled) to 0.078 (raw); DenSOINN reaches NMI 0.161 ± 0.012 raw, matching the paper's 0.146 ± 0.029 (full dataset), without any preprocessing.

My main takeaway is that the self-adaptive metric is not a feature of DenSOINN — it is its core identity. What makes it genuinely different from earlier algorithms in this course is that it never asks you to preprocess the data; the metric adapts to the distribution on its own, making it the first algorithm here I would run comfortably on truly raw data.

The Alpha–Gamma Observation. While tuning DenSOINN I noticed that with the right γ and α, the *connected components of the online network alone* can serve as a rough macro-cluster estimate, without running the offline phase. γ (denoising threshold) controls how aggressively low-weight bridge nodes are pruned: a high enough γ removes the sparse connective tissue between true clusters, causing the network to fragment into clean, well-separated components — one per cluster. At that point each component is already a macro-cluster, and setting α (density-peak sensitivity) high enough forces the offline phase to declare just one center per component rather than fragmenting it further. The first screenshot pair in Figure 5.6 shows this on a well-separated

dataset: 6 components emerge directly, matching the macro-cluster output almost exactly.

This shortcut only works when the true clusters are spatially separated and γ can actually cut the bridges between them. On a dense, overlapping dataset like RBF3, the online network tends to merge into a single large component regardless of γ — as shown in the second screenshot (1000 nodes, 1 component, 35 density-peak centers needed to recover structure). In such cases the full offline phase is essential, and the component structure gives no useful clustering signal on its own. The observation made me think of the online graph as more than just intermediate state: on the right data, it is already a clustering.

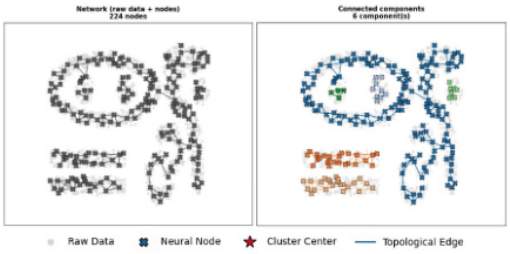


Figure 5.4: * **Well-separated data (Complex9):** 6 clean components emerge after denoising, each corresponding to one macro-cluster.

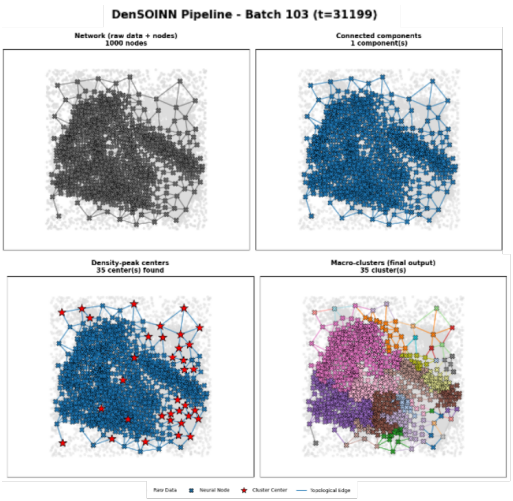


Figure 5.5: * **Dense overlapping data (RBF3-like):** 1 component despite tuning; 35 density-peak centers are needed to recover the macro-cluster structure.

Figure 5.6: Alpha–Gamma observation. On clean data, components are a viable cluster proxy (left). On dense data, the offline phase is unavoidable (right).

Chapter 6

Block 4: LiarTree — Anytime Clustering with Noise

6.1 Algorithm Overview

LiarTree [6] extends ClusTree [5], a hierarchical stream clustering algorithm that organizes micro-cluster summaries — Cluster Features $(n, \mathbf{LS}, \mathbf{SS})$ — in a balanced, R-tree-style tree. Points descend from root to leaf by following the closest entry at each level, with CFs updated lazily along the way. The key property is **anytime insertion**: if the budget (wall-clock time or a distance-computation quota) runs out mid-descent, the point is parked in the buffer of the last entry reached and carried further by the next passing point as a *hitchhiker*. Insertions can be interrupted at any moment without corrupting the model. At query time, DBSCAN [4] is run on the leaf-level CFs to form macro-clusters.

ClusTree has three weaknesses on real-world streams. It absorbs every point, including noise, into the nearest entry — corrupting cluster summaries. New clusters can only emerge bottom-up from leaf level, making novelty slow to detect. And overlapping inner entries can misguide the descent. LiarTree targets all three with dedicated extensions: a **per-node noise buffer** that holds suspected outliers and promotes them to real clusters once enough evidence accumulates; **liar nodes** that grow a new cluster subtree one level per incoming point after graduation; and a **local overlap look-ahead** that redirects the descent when the two closest entries overlap, reducing future collision.

In our group of five, **Nils** built the core ClusTree structure and insertion loop. **I** took on the noise handling — noise probability scoring, the per-node noise buffer, and the Gompertz-based graduation event. **Anton** implemented the liar-node novelty mechanism. **Paul** implemented the overlap look-ahead. **Jack** led the evaluation.

6.2 My Contribution: Noise Handling

My task was to implement all of Section 2.3 of the paper: the noise probability check, the per-node noise buffer, and the noise-to-cluster graduation event. I structured this into a pure-math module `noise_handler.py` (no tree state) and the integration hooks inside `liartree.py`.

The Noise Probability Check

The noise probability of a point o at an inner node is:

$$\mathrm{np}(o) = \min_{e \in \text{node}} \frac{\mathrm{dist}(o,\, \mu_e)}{r_e}$$

A value below 1 means o sits within at least one entry's radius; a value at or above 1 means it is outside all of them. When $\mathrm{np}(o) \geq \theta$ the point is classified as noise and diverted to the node's noise buffer; otherwise it continues descending normally (Figure 6.1).

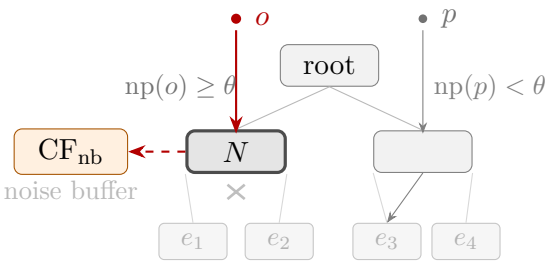


Figure 6.1: Noise check at an inner node N. Outlier o scores $\mathrm{np}(o) \geq \theta$ and is redirected to the per-node noise buffer $\mathrm{CF}_{\mathrm{nb}}$, blocked from descending further. Normal point p passes the check and continues to a leaf.

The check is restricted to **inner nodes**: leaf entries have near-zero or single-point radii that make the ratio meaningless, whereas inner entries aggregate entire subtrees and carry stable radii. The implementation reuses distances already computed during entry selection, so the noise check adds zero marginal cost.

The Noise Buffer and Graduation

Each inner node holds one CF-valued noise buffer (`node.noise_buffer`), a single micro-cluster $(n, \mathbf{LS}, \mathbf{SS})$ initialized lazily. When a point is buffered, the CF is first decayed by $2^{-\lambda \Delta t}$, then the new point is added, accumulating evidence in constant $O(d)$ memory.

After each update, the graduation test checks whether the buffer has earned the right to become a real cluster entry. All three conditions must hold simultaneously:

$$\underbrace{\texttt{enc_split} = \texttt{True}}_{\text{(1) structural trigger}} \quad \wedge \quad \underbrace{g(n_{\text{nb}}) \to 1}_{\text{(2) maturity gate}} \quad \wedge \quad \underbrace{g(n_{\text{nb}}) \cdot \rho_{\text{nb}} \;\geq\; \rho_{\text{avg}}}_{\text{(3) density test}}$$

(1) enc_split is set during descent whenever an empty or stale entry is found, signalling concept drift and structural room for a new cluster. Without this gate, graduation could fragment a stable, fully occupied tree.

(2) The **Gompertz gate** $g(n) = \exp(-b\,e^{-cn})$ is fitted from two anchors: $g(2) \approx 10^{-4}$ (two-point buffer is almost certainly noise) and $g(n_{\text{avg}}) \approx 0.97$ (buffer weight matches average entry). Parameters b and c are solved analytically, making the maturity threshold self-calibrating to stream density.

(3) The **density test** checks $g(n_{\text{nb}}) \cdot \rho_{\text{nb}} \geq \rho_{\text{avg}}$: the noise buffer must be at least as dense as the node's existing entries.

When all three hold, the buffer is promoted to a real entry (flagged `is_noise_promoted = True`), an empty liar-root subtree is attached for Anton's mechanism to grow, and the buffer resets to `None`. I also overrode `_leaf_proc` so `enc_split` forces a split rather than a merge at overflow, biasing toward finer resolution in regions where drift was detected.

6.3 Challenges, Design Decisions & Observations

Premature Noise Firing — An Implementation Gap Not in the Paper

The most significant problem I encountered was one the paper does not mention: with any noise threshold $\theta < 1.0$, every single point was classified as noise from the moment the tree grew its first inner node, producing $k = 0$, $F_1 = 0$ across all datasets.

Recall the formula:

$$\text{np}(o) = \min_{e \in \text{node}} \frac{\text{dist}(o,\, \mu_e)}{r_e}$$

This formula is only valid when r_e reflects a stable single-cluster region. The first inner node forms at $t \approx L{+}1$ (where L is the leaf capacity), and at that point entries hold just $n = 1$–2 points. With $n = 1$: $r_e = 0$, the old

code skipped the entry, and returned np = 1.0 when *all* were skipped. With $n = 2$ from different clusters: the radius spans the inter-cluster gap ($r_e \approx 0.1$, $d \approx 3.0$, $d/r \approx 30$, capped to 1.0). Either way $\text{np}(o) \geq \theta$ for any threshold below 1.0.

This creates a deadlock: noise fires, the tree never grows, entries never become stale, `enc_split` stays `False`, and graduation is permanently blocked. The fix (Figure 6.2) is a **maturity gate**: skip entries with $n <$ `noise_min_weight` (default 5.0) and return np = 0.0 when no valid entries remain — no valid comparison means the point is not noise. Two lines of change:

```
# old:  no valid entries → return 1.0  (noise unconditionally)
if e.summary.n < self.noise_min_weight:
    continue                                # skip immature entries
...
if not np.isfinite(min_ratio):
    return 0.0                              # no valid entries → not noise
```

The value 5.0 is not a sensitive tuning parameter; it holds the noise check dormant until entries represent mature, single-cluster regions. This gap is entirely absent from the paper.

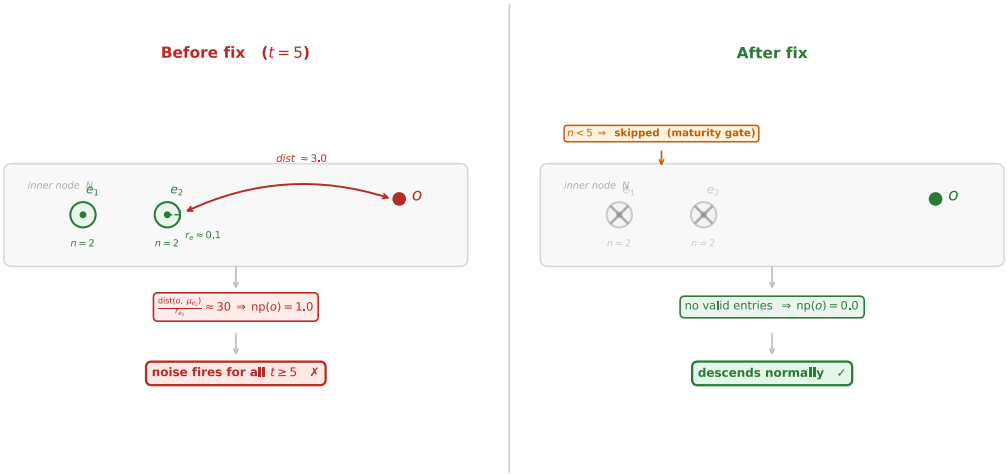


Figure 6.2: Left: at $t = L{+}1$ the immature inner entries (each $n = 2$) produce $\text{np}(o) = 1.0$, causing noise to fire for all subsequent points. Right: after the fix, entries with $n < 5$ are skipped; no valid entries gives np = 0.0 and the point descends normally.

The `is_noise_promoted` Flag

After graduation, the promoted entry has a very compact CF, so the noise check would immediately misclassify genuine members of the growing cluster as noise, starving the liar subtree. The fix is a persistent `is_noise_promoted` flag on Entry — set at graduation, never cleared — that excludes the entry from np computations. It must be persistent because the liar mechanism clears `is_liar` and `is_liar_root` once the subtree reaches leaf depth, erasing all other memory of graduation.

Gompertz Numerical Stability

When n_{avg} is barely above 2, the slope parameter c becomes large and $b = -\ln(t_0) \cdot e^{2c}$ overflows to `inf`, causing $\exp(-b \cdot e^{-cn})$ to evaluate as $\infty \cdot 0 =$ `nan`. The fix is to substitute b back into the exponent algebraically:

$$g(n) = \exp\big(\ln(t_0) \cdot e^{c(2-n)}\big)$$

For $n \geq 2$ the inner exponent $c(2-n) \leq 0$, so this form stays well-defined for any value of c, while still hitting the exact anchor values at $n = 2$ and $n = n_{\text{avg}}$.

Design Choices and What the Paper Leaves Open

Several design decisions arose from gaps in the paper's specification.

Inner nodes only. Algorithm 1 places the noise check "before the leaf check," which is technically ambiguous. Restricting it to inner nodes was necessary: leaf entries can have $r_e = 0$ or cover a handful of points, making the ratio d/r meaningless.

CF buffer. A single CF vector $(n, \mathbf{LS}, \mathbf{SS})$ gives $O(d)$ memory regardless of how many points accumulate, and all three graduation statistics are recoverable in $O(1)$.

Distance reuse. Every $\text{dist}(o, \mu_e)$ used in the noise check was already computed by the entry selection step. Passing the pre-ranked list directly adds zero extra distance calls — the noise check costs nothing on a budget-limited run.

Offline clustering level. Playing with the visualization tool hosted on: `stream-viz.birdiegang.uk`, which lets you zoom through the tree level by level, raised an unexpected question: does DBSCAN have to run on the leaves? At inner levels the CF summaries already show a coarse but recognizable cluster structure. Running DBSCAN there would be faster (far fewer CFs) and might be sufficient when clusters are well-separated. The zoom feature made

this gap visible in a way raw metrics never do — you can watch cluster boundaries sharpen level by level and judge where "good enough" starts. Whether and how to exploit this remains an open question for future work.

Bibliography

[1] A density-based competitive data stream clustering network with self-adaptive distance metric. *Neural Networks*, 110:141–158, 2019.

[2] Feng Cao, Martin Ester, Weining Qian, and Aoying Zhou. Density-based clustering over an evolving data stream with noise. In *Proceedings of the 2006 SIAM International Conference on Data Mining (SDM)*, pages 328–339, 2006.

[3] Yixin Chen and Li Tu. Density-based clustering for real-time stream data. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '07, page 133–142, New York, NY, USA, 2007. Association for Computing Machinery.

[4] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. pages 226–231, 1996.

[5] Philipp Kranen, Ira Assent, Corinna Baldauf, and Thomas Seidl. The clustree: indexing micro-clusters for anytime stream mining. *Knowledge and Information Systems*, 29(2):249–272, 2011.

[6] Philipp Kranen, Felix Reidl, Fernando Sanchez Villaamil, and Thomas Seidl. Hierarchical clustering for real-time stream data with noise. In Judith Bayard Cushing, James French, and Shawn Bowers, editors, *Scientific and Statistical Database Management*, pages 405–413, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[7] James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. Oakland, CA, USA, 1967.

[8] Mustafa Tareq, Elankovan A. Sundararajan, Masnizah Mohd, and Nor Samsiah Sani. Online clustering of evolving data streams using a density grid-based method. *IEEE Access*, 8:166472–166490, 2020.

Declaration of Authorship

This report was authored by me as part of the practical course "Practical: Stream Clustering" at Ludwig Maximilian University (LMU).
The author acknowledges the use of artificial intelligence tools (Google Gemini) for linguistic enhancements, including grammar checks, the improvement of paragraph and sentences structures and transitions during the preparation of this report. This tool was however *NOT* used for the generation of ideas or completely new content. All sources used are acknowledged as references.

Munich, 01.08.2026



.....
Felix Hajuj